Дисципліна «Технології DevOps»

ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ №3

Тема: «Розгортання власного застосунку в Kubernetes»

Студент	Короленко Влад Романович (vlkorolenko)
Репозиторій проєкту	https://github.com/vlkorolenko/devops-lab1
Стек застосунку	DevOps TechStore: Spring Boot (product-service, order-service), PostgreSQL 16, Nginx SPA
Середовище Kubernetes	Minikube v1.39.0 (driver=docker), Kubernetes v1.37.0, Namespace: devops-techstore

1. Підготовка версіонованих образів та кластер Minikube

Для проєкту DevOps TechStore (product-service, order-service, frontend) у Dockerfiles додано аргумент збірки APP_VERSION та оновлено CI/CD пайплайн (.github/workflows/ci.yml) для публікації мультиархітектурних образів (linux/amd64, linux/arm64) у GHCR за релізними тегами v1.0.0 та v1.1.0 без використання тегу latest.

Локальний кластер Kubernetes розгорнуто за допомогою Minikube (драйвер docker, Kubernetes v1.37.0). Перевірено стан вузла, системних подів у kube-system, увімкнено доповнення metrics-server та ingress, а також завантажено всі версіоновані образи (1.0.0, 1.1.0 та postgres:16-alpine) у Minikube.

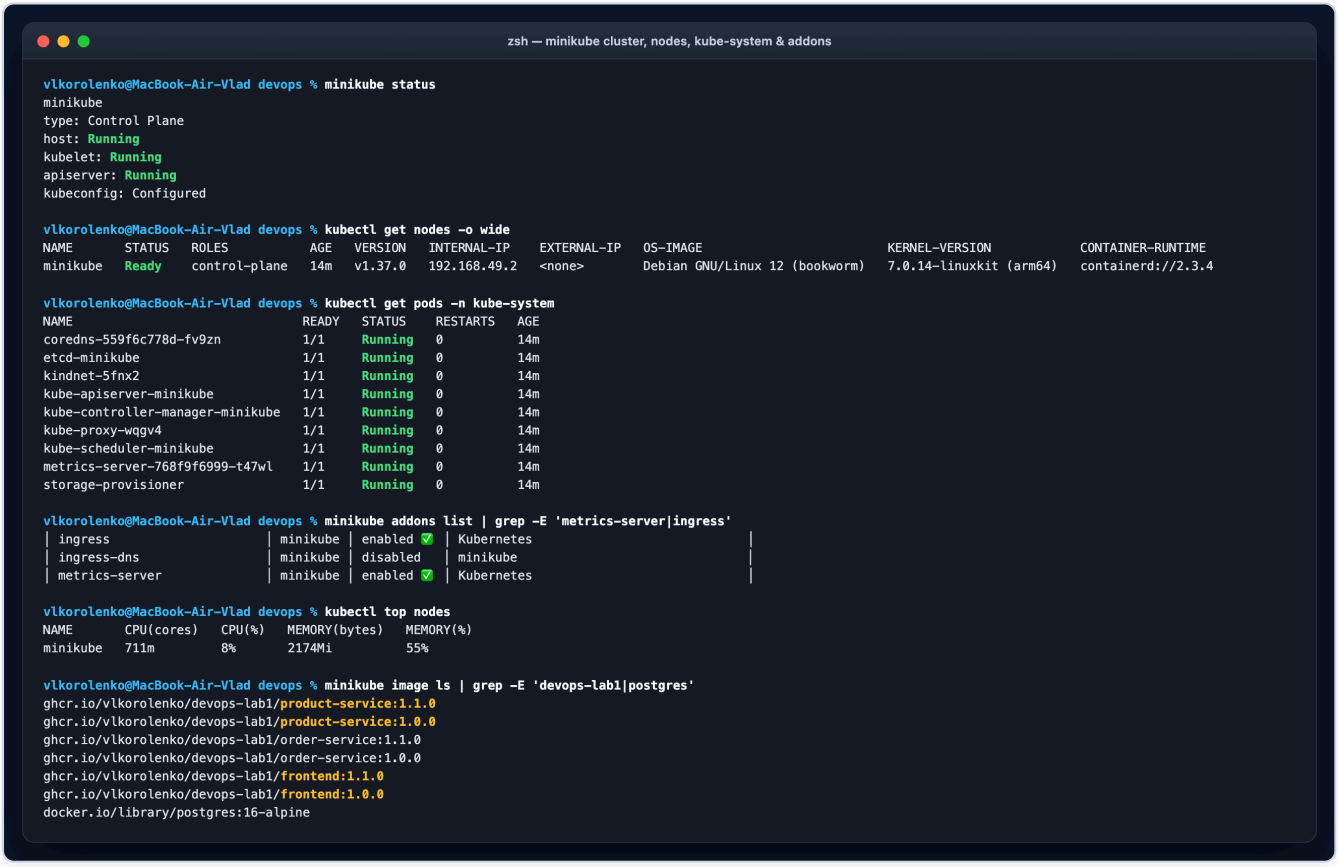


Рис. 1.1 — Стан кластера Minikube, вузол, системні поди kube-system, аддони metrics-server / ingress та список версіонованих образів.

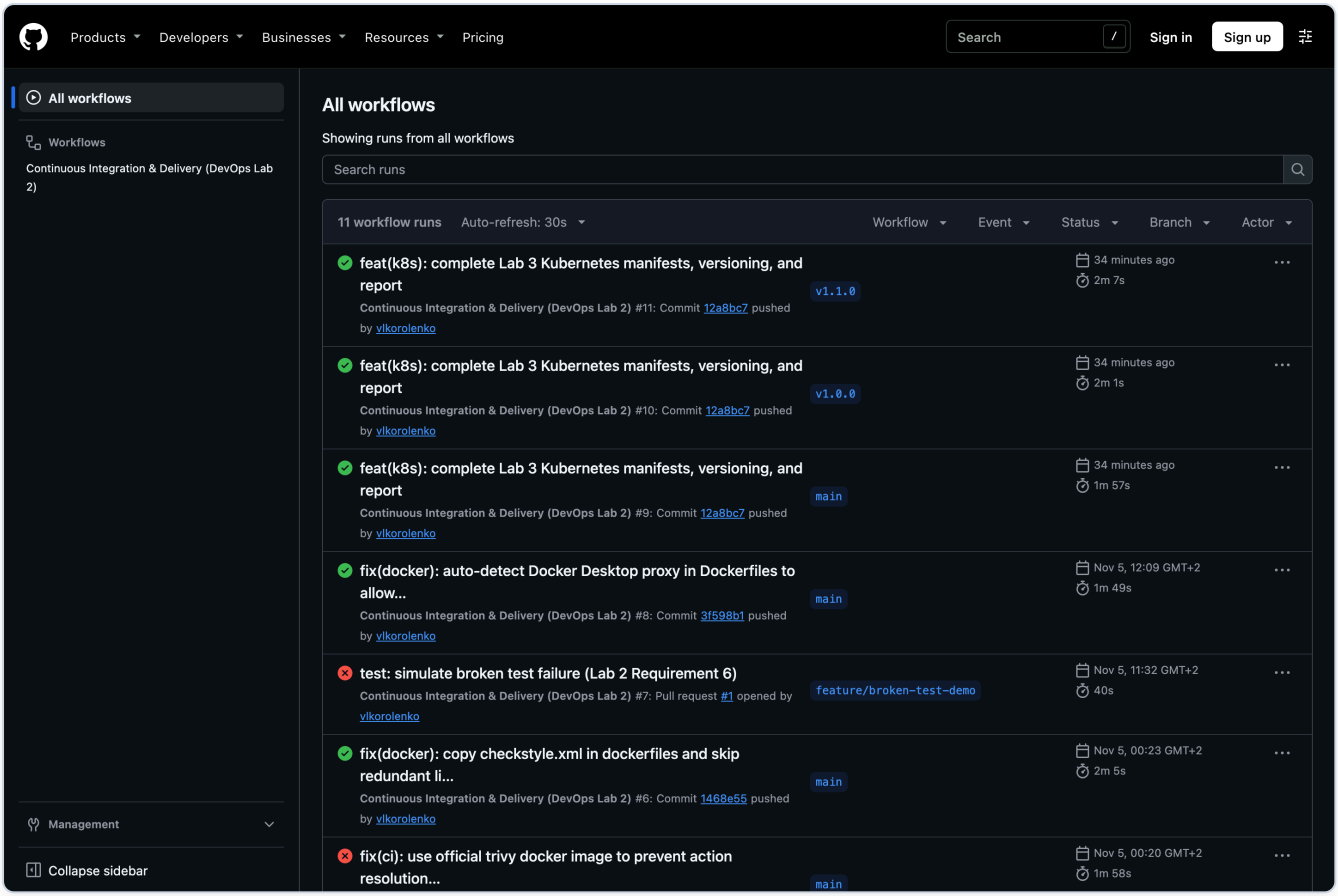


Рис. 1.2 — Успішне виконання CI/CD пайплайна GitHub Actions для гілки main та релізних тегів v1.0.0 і v1.1.0 із публікацією в GHCR.

2. Namespace, маніфести Kubernetes (k8s/) та розгортання однією командою

Усі об'єкти ізольовано в окремому просторі імен devops-techstore (k8s/00-namespace.yaml). У каталозі k8s/ вручну створено 13 маніфестів за принципом «один файл — один об'єкт»:
• ConfigMap (k8s/01-configmap.yaml) — 7 env-параметрів та змонтований JSON-файл store-config.json;
• Secret (k8s/02-secret.yaml) — тестові облікові дані PostgreSQL та сервісний ключ INTERNAL_API_KEY;
• PersistentVolumeClaim (k8s/03-postgres-pvc.yaml) + PostgreSQL Deployment/Service (ClusterIP :5432);
• product-service та order-service Deployments (по 2 репліки, :1.0.0, requests/limits, HTTP readiness/liveness проби) + ClusterIP Services;
• frontend Deployment (2 репліки, :1.0.0, Nginx reverse proxy + монтування store-config.json) + NodePort Service (30080) + Ingress (nginx).

Весь стек розгортається з нуля на чистому кластері однією командою kubectl apply -f k8s/ (або make k8s-up).

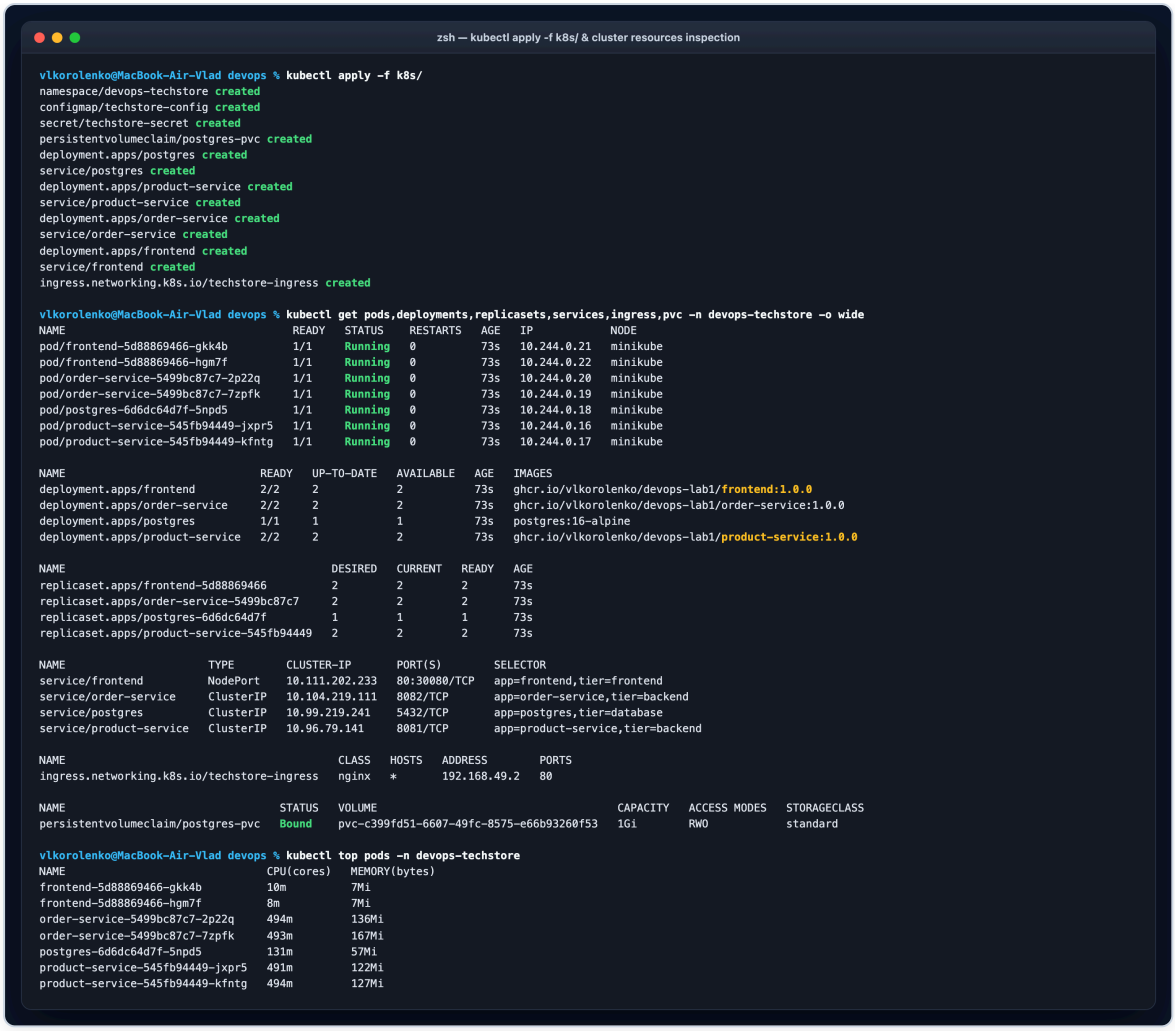


Рис. 2.1 — Розгортання маніфестів командою kubectl apply -f k8s/, перелік Pod / Deployment / ReplicaSet / Service / Ingress / PVC та метрики kubectl top pods.

3. Перевірка веб-застосунку в браузері, ConfigMap та Secret

Застосунок відкрито у браузері Google Chrome через Ingress контролер: відображається каталог товарів із PostgreSQL (6 позицій), версія v1.0.0, ім'я пода, банер із ConfigMap («Лабораторна робота №3 • Kubernetes (minikube)») та індикатор стану «Всі сервіси UP». Оформлене замовлення #1 («Олена Коваленко», $1899.99, статус CONFIRMED) успішно зберігається в PostgreSQL та відображається в модальному вікні «Історія замовлень (Order Service)».

Також підтверджено потрапляння параметрів із ConfigMap та Secret всередину контейнерів через kubectl exec env, змонтований файл /usr/share/nginx/html/config/store-config.json та HTTP-відповідь /actuator/product-service/info.

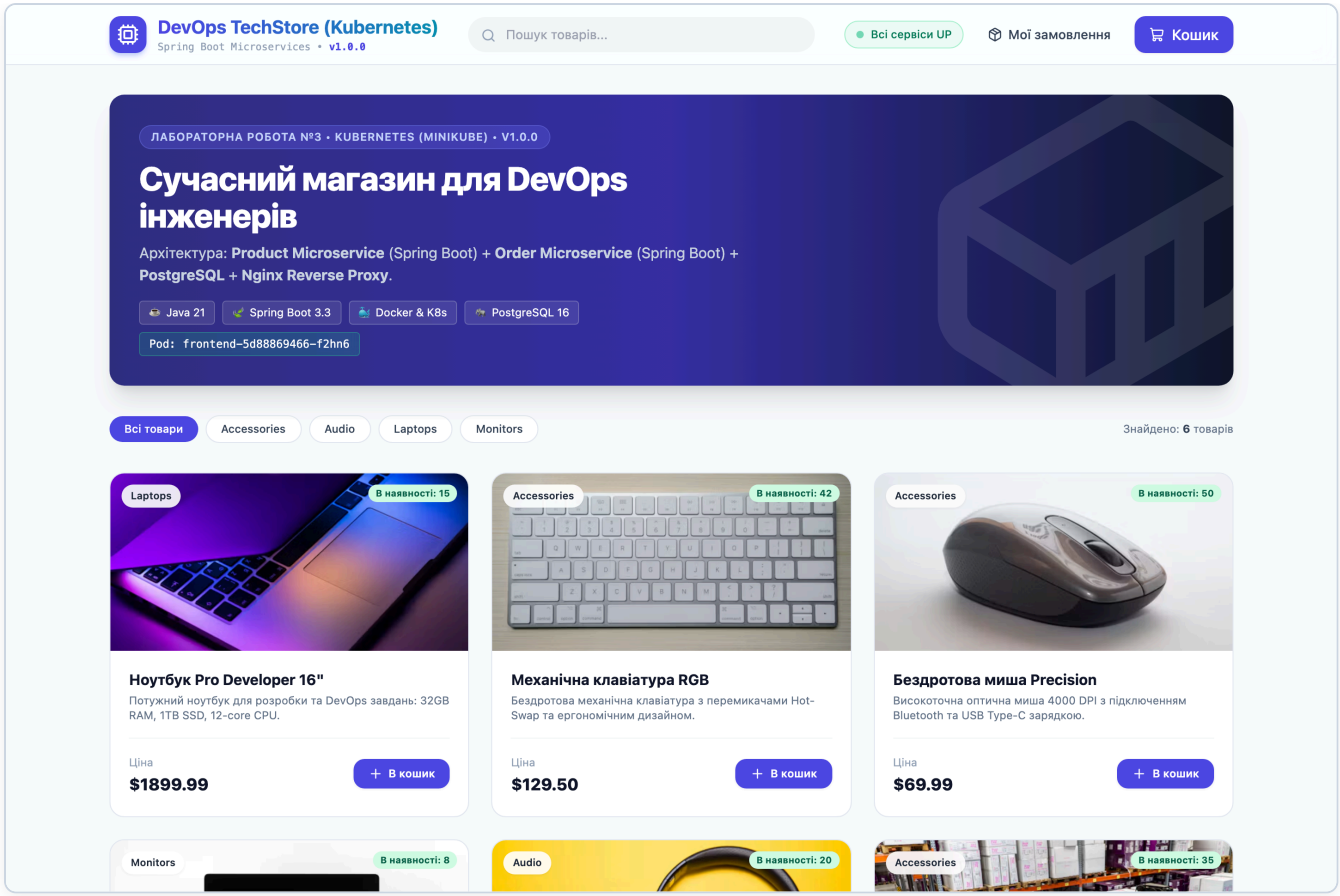


Рис. 3.1 — Робота веб-застосунку DevOps TechStore (v1.0.0) у браузері: каталог товарів із PostgreSQL, метадані пода та банер із ConfigMap.

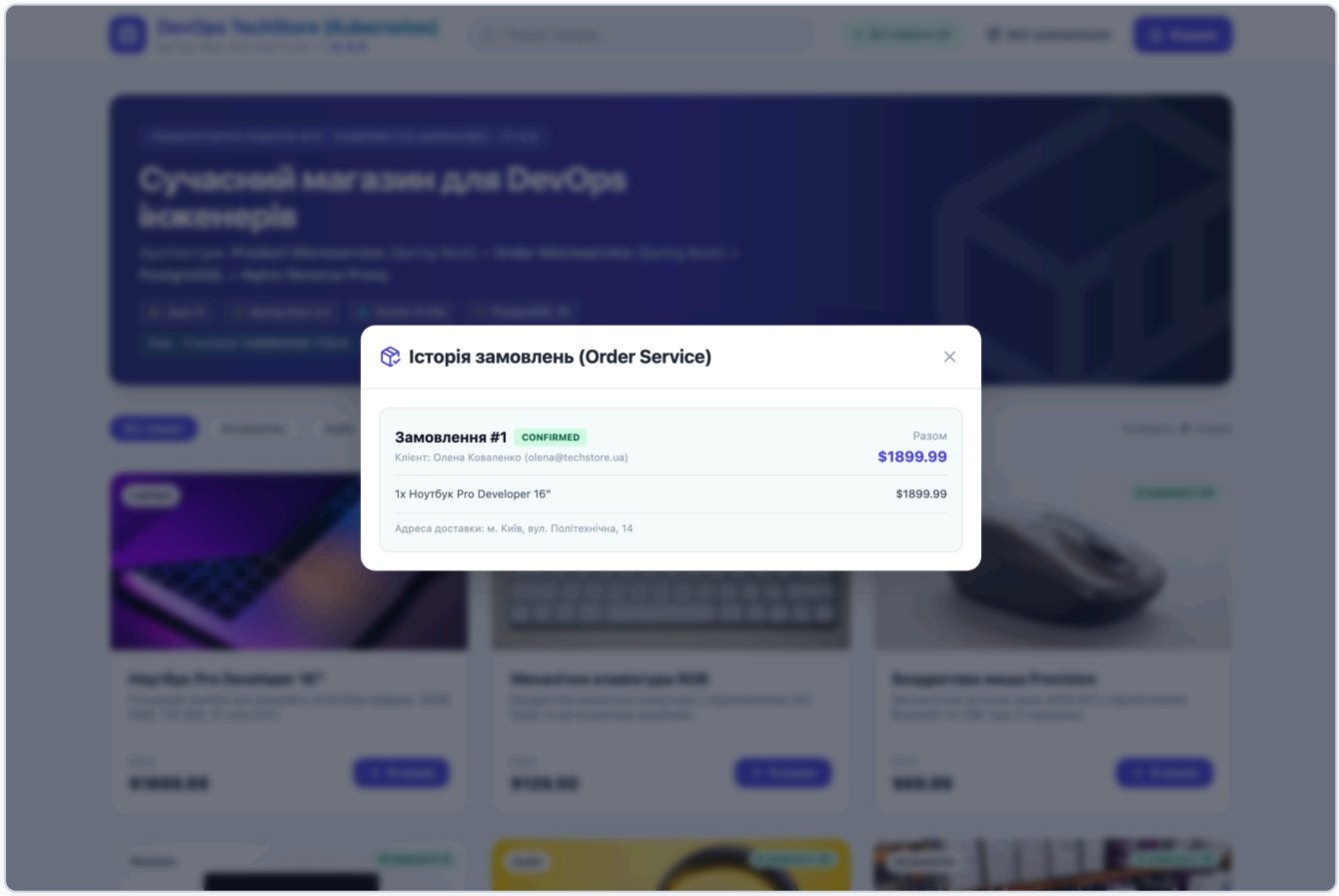


Рис. 3.2 — Модальне вікно «Історія замовлень (Order Service)» у браузері: підтверджене збереження замовлення #1 у PostgreSQL.

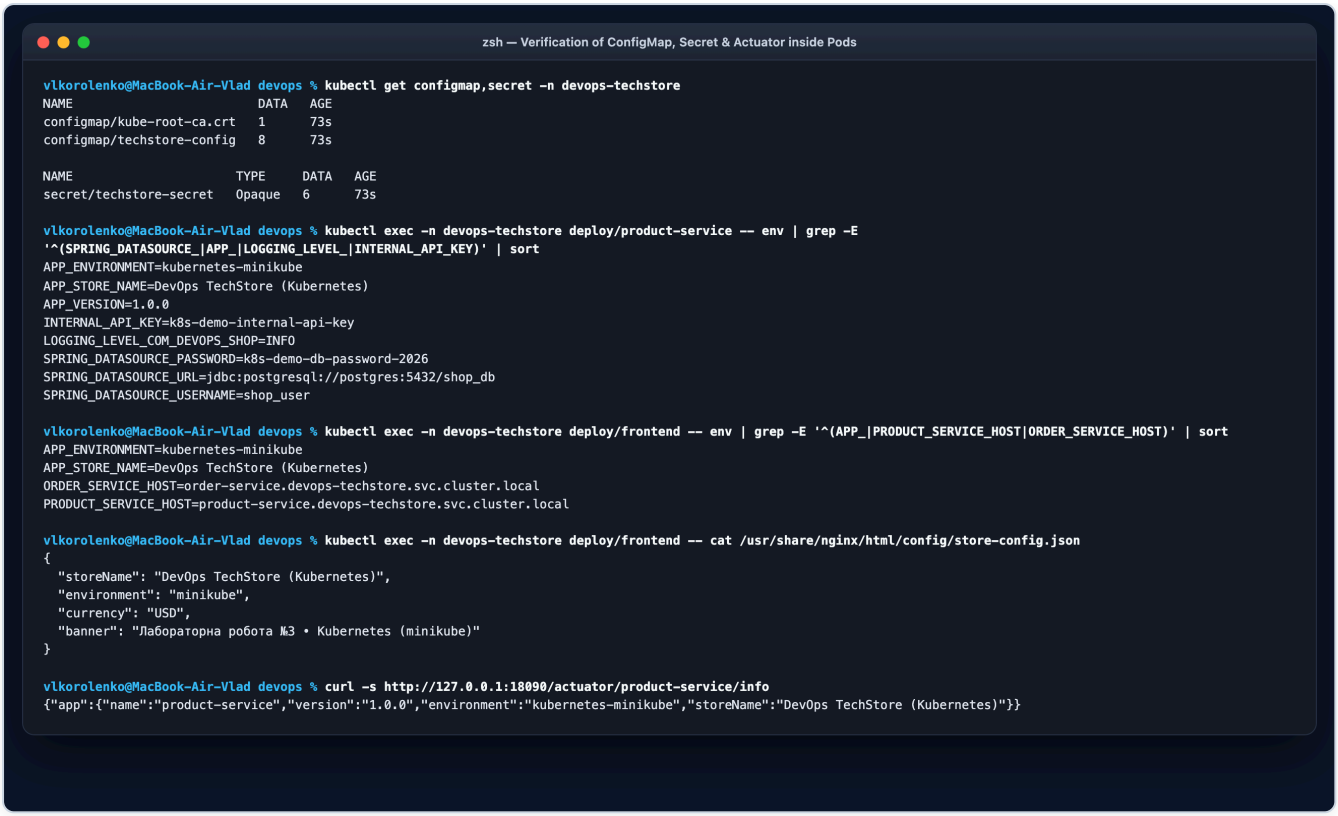


Рис. 3.3 — Перевірка змінних оточення ConfigMap/Secret, змонтованого файлу store-config.json та /actuator/product-service/info.

4. Керування життєвим циклом: самовідновлення та масштабування

Самовідновлення (Self-Healing): після примусового видалення одного з подів frontend (kubectl delete pod) контролер ReplicaSet миттєво створив новий под, автоматично відновивши бажану кількість реплік 2/2.

Масштабування та балансування: після виконання kubectl scale deployment/frontend --replicas=4 сервіси NodePort (30080, kube-proxy) та Ingress (80, ingress-nginx) рівномірно розподілили 20 HTTP-запитів між усіма чотирма подами (56ndm, f2hn6, f2ttt, tsn49).

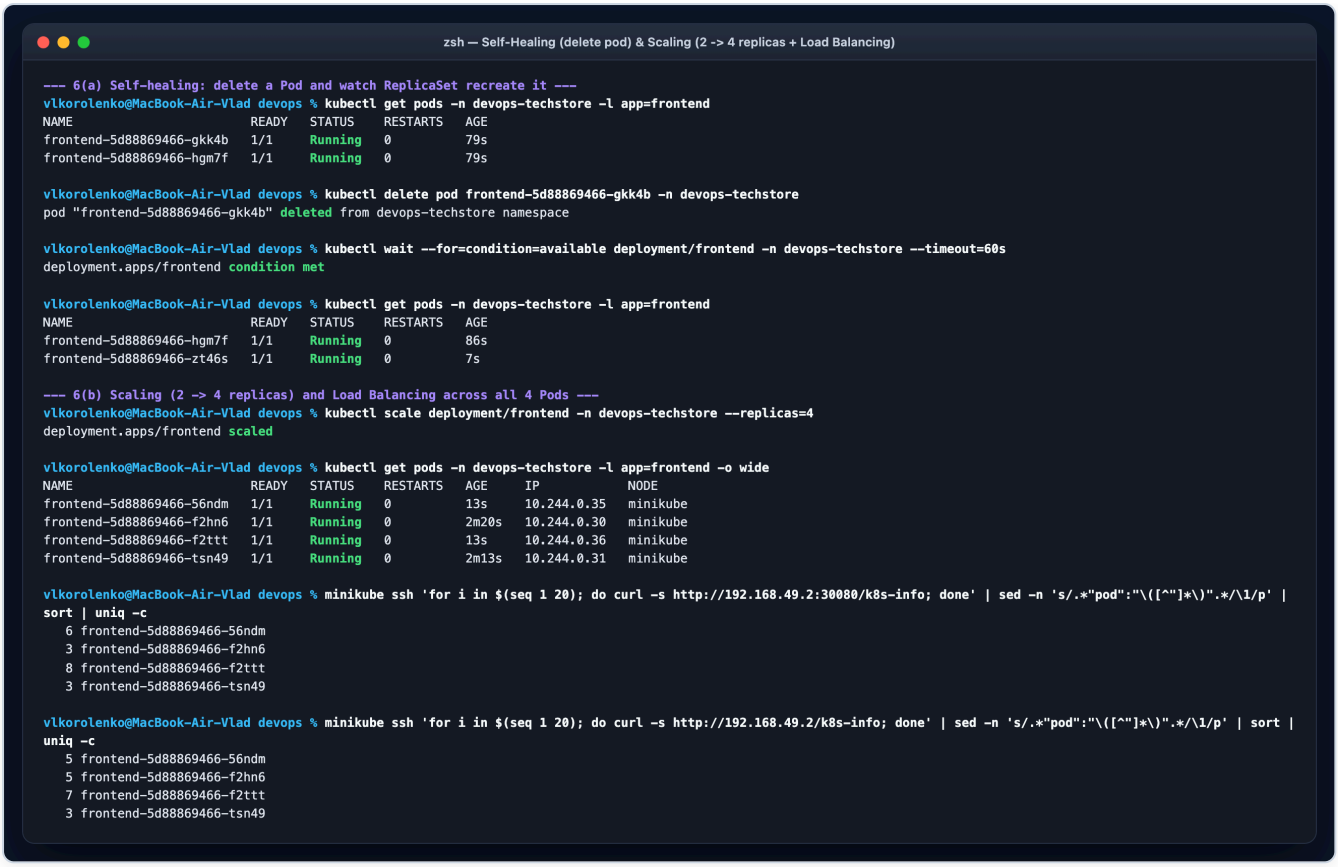


Рис. 4.1 — Демонстрація самовідновлення після видалення пода та балансування навантаження на 4 репліки після масштабування.

5. Rolling Update (1.0.0 → 1.1.0), Rollback та нульовий час простою

Стратегія RollingUpdate (maxUnavailable: 0, maxSurge: 1) у поєднанні з readinessProbe забезпечила оновлення frontend та product-service з версії 1.0.0 до 1.1.0 без жодної помилки доступності: під час оновлення безперервний HTTP-зонд зафіксував 33 із 33 успішних відповідей HTTP 200 OK (Failures=0). Після перевірки роботи нової версії v1.1.0 у браузері виконано перегляд історії (kubectl rollout history) та успішний відкат (kubectl rollout undo) назад до версії 1.0.0.

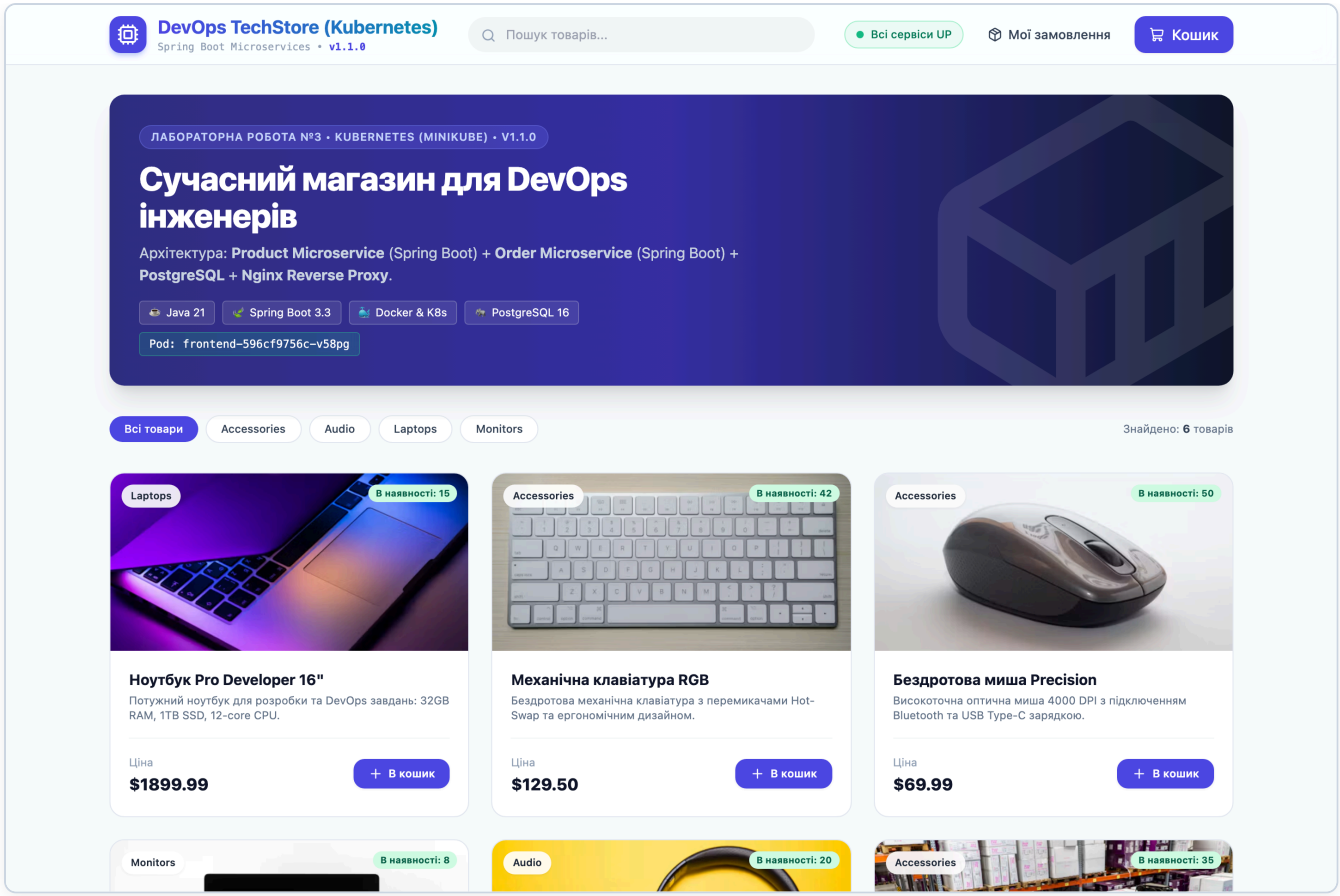


Рис. 5.1 — Вітрина DevOps TechStore у браузері після Rolling Update на версію v1.1.0.

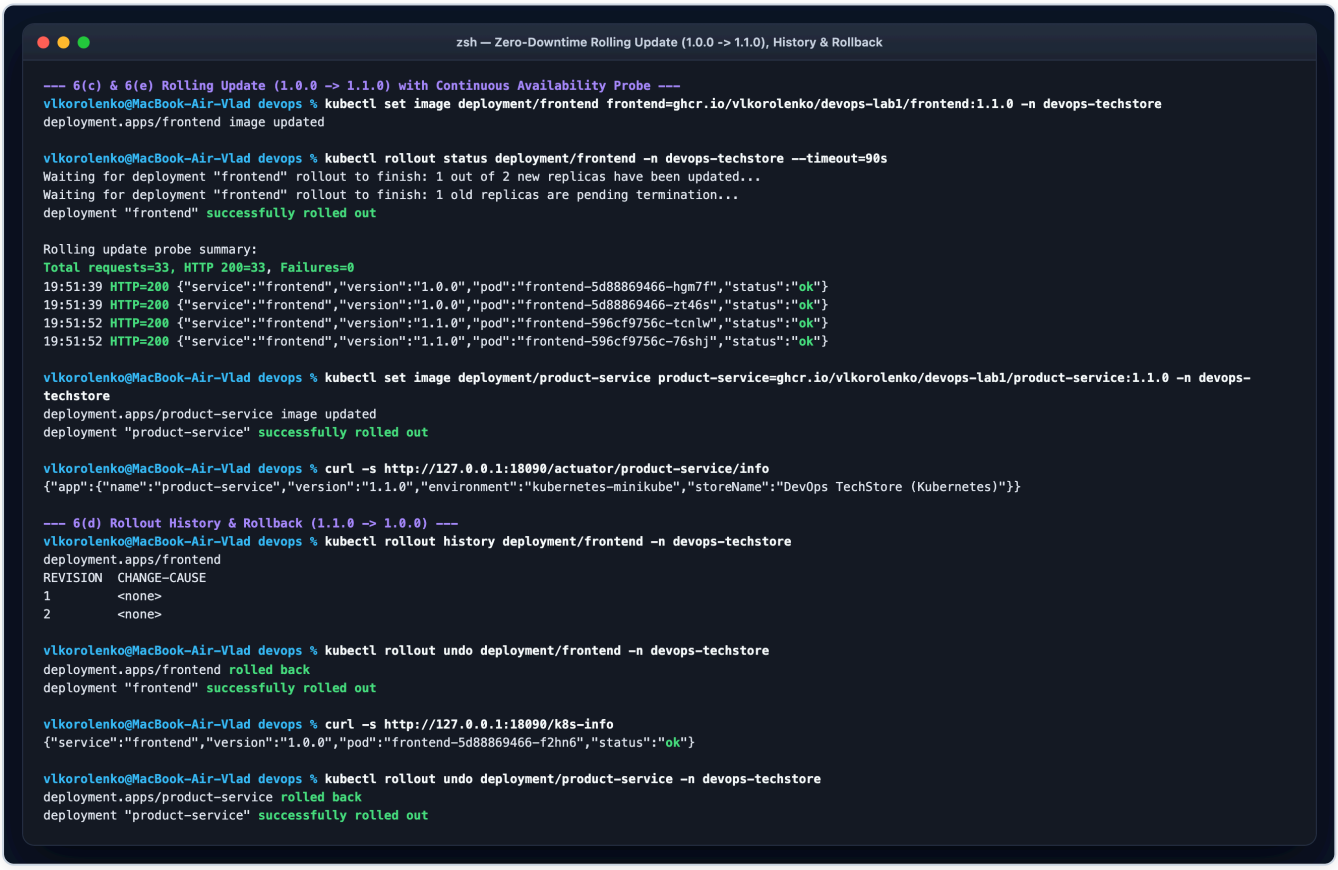


Рис. 5.2 — Процес Rolling Update 1.0.0 → 1.1.0, лог безперервного HTTP-зондування (33/33 HTTP 200) та Rollback до 1.0.0.

6. Діагностика помилок та відновлення кластера

Для демонстрації діагностики навмисно вказано неіснуючий тег образу frontend:9.9.9-nonexistent. У виводі kubectl get pods зафіксовано статус ErrImagePull/ImagePullBackOff (при цьому старі репліки 1.0.0 продовжували обслуговувати трафік), а команда kubectl describe pod у секції Events показала точну причину (not found).

Також виконано перегляд логів бекенду через kubectl logs, вхід у працюючий контейнер через kubectl exec -it, виправлення тегу назад на 1.0.0 та підтвердження повного відновлення всіх 7 подів (STATUS: Running, RESTARTS: 0).

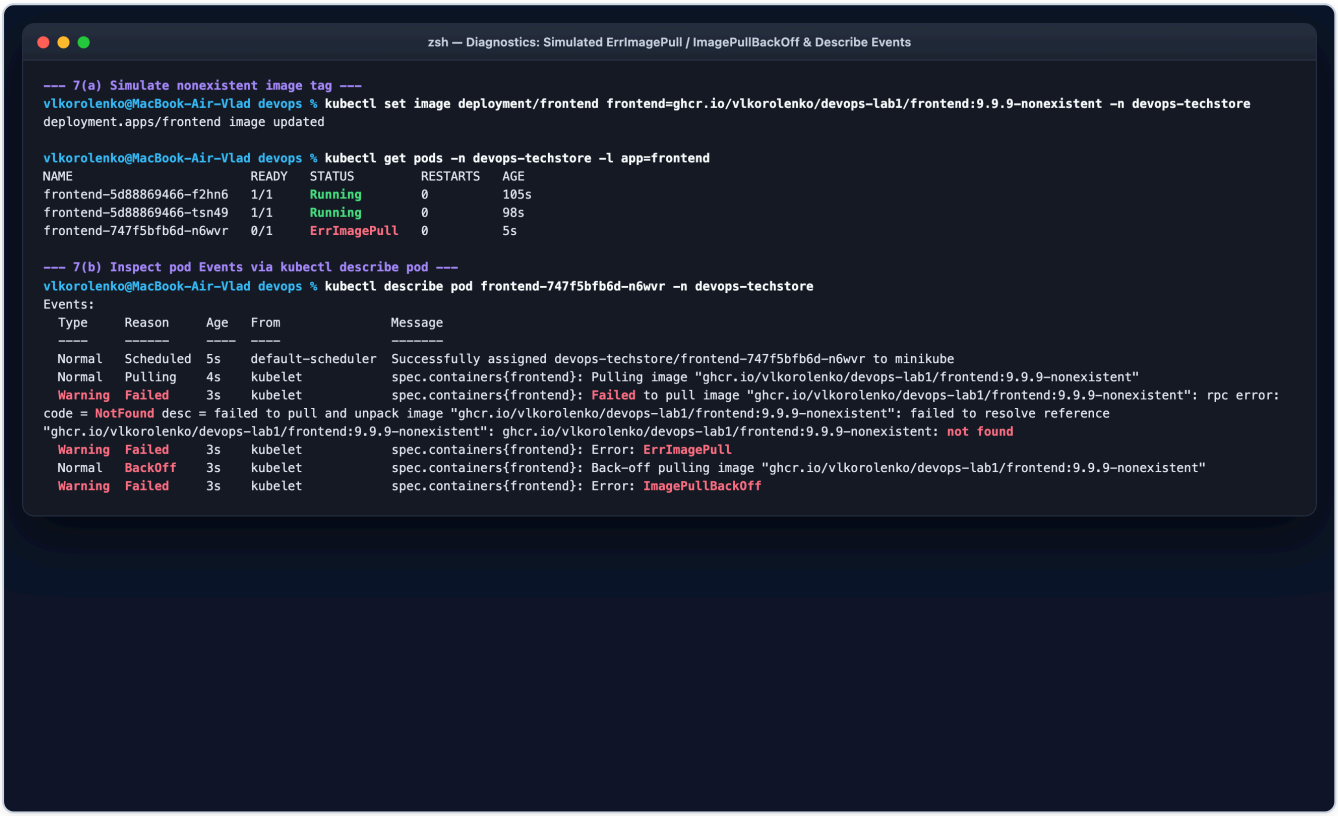


Рис. 6.1 — Симуляція неіснуючого тегу (ErrImagePull / ImagePullBackOff) та діагностика Events через kubectl describe pod.

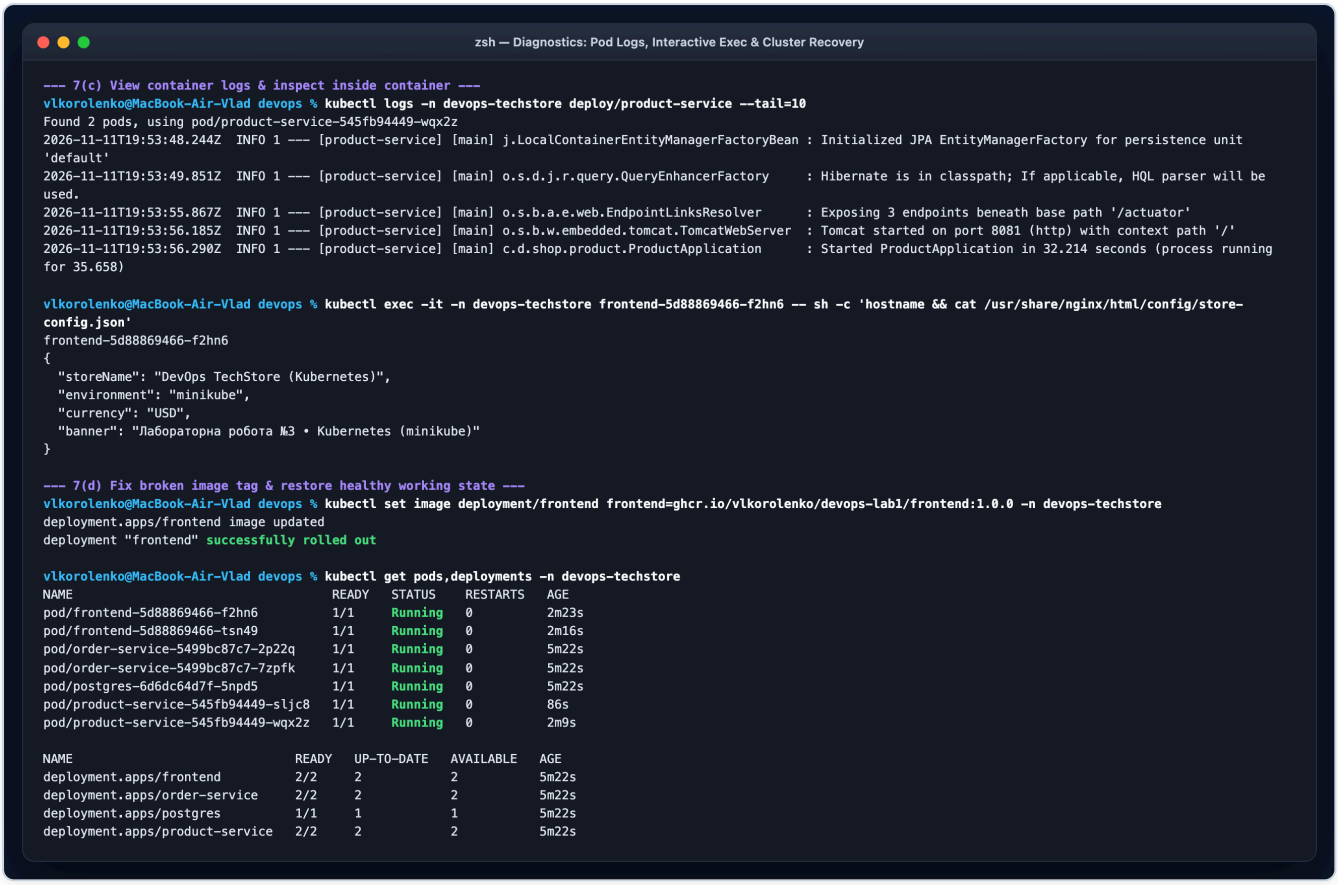


Рис. 6.2 — Перегляд логів (kubectl logs), інспекція контейнера (kubectl exec), виправлення образу та відновлення 2/2 Running.

Висновки

У ході лабораторної роботи мікросервісний веб-застосунок DevOps TechStore було повністю перенесено на маніфести Kubernetes (13 файлів у каталозі `k8s/`, окремий namespace `devops-techstore`, без використання тегу `latest`) та розгорнуто у локальному кластері Minikube однією командою. На практиці перевірено збереження стану БД PostgreSQL на PVC, передачу конфігурацій через ConfigMap і Secret, роботу вебінтерфейсу через Ingress та NodePort, самовідновлення після видалення пода, масштабування з балансуванням навантаження, безперебійний Rolling Update (1.0.0 → 1.1.0), відкат через Rollback та діагностику помилок ImagePullBackOff.